PROMPT PARA CLAUDE CODE — Diseño inicial del CRM "Greatstone Design"

> Pega este documento completo como primer mensaje en Claude Code, dentro de una carpeta local vacía (por ejemplo `greatstone-crm/`).

---

0. TU ROL Y ALCANCE (léelo primero)

Eres el diseñador/desarrollador front-end de la **fase 1 (diseño inicial)** de un CRM a medida. En esta fase construyes SOLO la interfaz, con **datos de ejemplo (mock)**. NO hay backend, NO hay base de datos, NO hay claves ni integraciones reales. Más adelante otro equipo conectará Supabase/APIs, así que **la arquitectura debe dejar un punto único donde se cambian los datos mock por datos reales** (ver sección 8).

Entregable: una app que se ejecuta en local (`npm install && npm run dev`) y se despliega en local (`npm run build && npm run preview`), con todos los módulos navegables y con aspecto profesional, lista para que el responsable la evalúe. Después de su valoración, tú afinarás el diseño para móvil y todas las resoluciones.

Trabaja de forma autónoma y entrega completo: primero planifica la estructura de carpetas y el sistema de diseño, luego construye módulo por módulo. No me pidas confirmación en cada paso; si algo es ambiguo, elige la opción más profesional y déjala anotada en `DECISIONS.md`.

---

1. LA CLIENTA

- **Negocio:** Greatstone Design — "Miami's Best Interior Design & Remodel" (greatstonedesign.com, sitio en Wix). Instagram: @greatstonedesign.
- **Contacto público:** Tanya (tanya@greatstonedesign.com), 786-586-7757. Trabaja en South Florida, centrada en Miami, "por cita", presencial y virtual.
- **Qué hace:** diseño de interiores de lujo, remodelación llave en mano (cocinas, baños, closets, pisos), decoración y *home staging*, planificación de espacios, selección de mobiliario y acabados a medida, y experiencia en cumplimiento de normas de condominios (HOA/condo compliance). Mucho trabajo en condos de Coconut Grove y similares.

- **Posicionamiento/tono:** boutique, trato 1:1, elegante, cálido y personal ("Elegant interiors. Stress-free remodels. One-on-one attention." / "We Treat Every Client Like Family"). Declara 95 % de clientes por referido y ~35 socios/proveedores de confianza.
- **Su formulario de presupuesto pide:** nombre, apellido, email, teléfono, cuándo quiere empezar, y descripción del proyecto. Usa estos mismos campos como base del formulario de Lead.

Implicación para el CRM: los **referidos y los proveedores/partners** son importantes (modelarlos), los proyectos son de **diseño + obra** (habitaciones, presupuestos, selecciones de materiales, visitas), y el cliente final es residencial de gama alta.

---

2. STACK Y REGLAS TÉCNICAS

- **JavaScript** (sin TypeScript). Usa **React + Vite en JavaScript (JSX)** con **React Router**. Sin frameworks de UI pesados (no MUI/Ant/Bootstrap): CSS propio con **variables CSS (design tokens)**. Se permiten librerías pequeñas: `lucide-react` (iconos), una librería de gráficos ligera (Recharts o Chart.js), `date-fns`, y un drag-and-drop ligero para el Pipeline (`@dnd-kit/core` o similar).
- Código modular, componentes reutilizables, sin duplicar estilos.
- Despliegue final previsto: **GitHub → Vercel**. Deja `vercel.json` con el rewrite para SPA (todas las rutas a `index.html`). En esta fase solo ejecútalo en local.
- Inicializa `git` en la carpeta, con commits pequeños y claros por módulo. No subas nada a ningún remoto.
- Crea `README.md` (cómo ejecutar), `DECISIONS.md` (decisiones y supuestos) y `STRUCTURE.md` (mapa de carpetas y rutas).
- **Prohibido:** claves reales, llamadas a APIs externas, datos personales reales. Todos los nombres, teléfonos y correos del mock son ficticios.

---

3. SISTEMA DE DISEÑO

Tipografía

Inter en toda la app (cargada localmente vía `@fontsource/inter` o `font-face` propio, no desde CDN). Pesos 400/500/600/700. Activa `font-feature-settings: "tnum"` en cifras de tablas/finanzas. Escala tipográfica definida en tokens (12/13/14/16/20/24/32).

Colores — IMPORTANTE

No pude extraer los hex exactos de la web de la clienta. **Tu primera tarea de diseño es obtenerlos tú mismo:** abre https://www.greatstonedesign.com y su Instagram en el navegador, usa DevTools / un cuentagotas y anota en `DECISIONS.md` los colores reales (fondo, texto, acento, botones, logo) y la tipografía de su web. La web usa botones en MAYÚSCULAS y un logotipo tipográfico "Greatstone." (minúscula con punto), con estética elegante y neutra.

Mientras tanto, usa esta **paleta provisional** (piedra cálida + carbón + un acento sobrio), y define TODO como variables en `src/styles/tokens.css` para que cambiarla sea tocar un solo archivo:

```
:root{
  /* Neutros piedra (provisional) */
  --bg:#F7F5F2; --surface:#FFFFFF; --surface-2:#F0ECE6; --border:#E2DDD5;
  --text:#1F1D1B; --text-muted:#6B665F;
  /* Marca (provisional: sustituir por los colores reales de la clienta) */
  --brand:#1F1D1B; --brand-contrast:#FFFFFF;
  --accent:#A68A64;  --accent-soft:#EFE6D8;
  /* Estados */
  --success:#2F7D5B; --warning:#B7791F; --danger:#B3402F; --info:#2F6690;
  /* Forma */
  --radius-sm:6px; --radius:10px; --radius-lg:14px;
  --shadow-1:0 1px 2px rgba(31,29,27,.06); --shadow-2:0 8px 24px rgba(31,29,27,.1
}
```

Soporte de **modo oscuro** opcional vía `[data-theme="dark"]` con los mismos tokens (si hay tiempo; prioridad baja). Contraste AA mínimo en todo texto.

Estilo general

Limpio, aireado, profesional, con jerarquía clara. Tarjetas con borde fino y sombra mínima, tablas densas pero legibles, estados vacíos cuidados, *skeletons* de carga, *toasts* para confirmaciones. Iconografía coherente (lucide). Logo provisional: wordmark "Greatstone." en texto.

Idioma de la interfaz

Interfaz en **inglés** (clienta en Miami), preparada para i18n: todos los textos en un diccionario `src/i18n/en.js` (y deja `es.js` creado con las mismas claves), sin textos sueltos dentro de los componentes.

---

4. PATRÓN DE INTERACCIÓN HÍBRIDO (CLAVE DEL

PROYECTO)

La app NO debe abrir todo en modal ni todo en página. Es **híbrida**:

4.1 Cuándo usar MODAL (solo si es imprescindible)

Acciones cortas y autocontenidas: confirmar/eliminar, crear o editar algo rápido (≤ ~6 campos: nueva tarea, nueva cita, nueva nota, añadir contacto rápido, registrar un pago), selectores/pickers, vista previa rápida.

4.2 Cuándo usar PÁGINA NUEVA (ruta propia)

Flujos largos o con mucho contenido: crear/editar Proyecto, Lead completo, ficha de Contacto, Estimate o Invoice (constructor de líneas), campaña, conversación ampliada, detalle de proyecto, cualquier formulario > ~6 campos o con secciones/tabs.

- Cada página nueva tiene **botón "← Back"** visible arriba a la izquierda que regresa **exactamente a donde se hizo clic**: misma pantalla, mismo scroll, mismos filtros/pestañas/paginación. Implementa con React Router (`navigate(-1)` + restauración de estado y scroll; guarda filtros en la URL con query params). Si se entra por enlace directo, "Back" lleva al listado padre.
- Breadcrumb discreto además del botón Back.

4.3 Estructura ÚNICA de formulario (modal y página comparten el mismo componente base)

Crea componentes `FormShell` / `ModalShell` / `PageShell` reutilizables y úsalos en TODA la app, de modo que **todos los formularios tengan la misma estructura y tamaño**:

- **Cabecera fija** (título, botón cerrar/Back) y **pie fijo** (botones Cancelar / Guardar, estado de guardado) que NO se mueven al hacer scroll. Solo el **cuerpo** hace scroll vertical.
- **Tamaños fijos y consistentes** del modal (p. ej. `sm` 480 px, `md` 640 px, `lg` 840 px; alto máximo `min(90vh, 760px)`); el mismo tamaño para el mismo tipo de formulario, siempre.
- **Nunca scroll horizontal.** El contenido se adapta; las rejillas de campos colapsan de 2 columnas a 1 en anchos pequeños. Textos largos hacen wrap, tablas dentro de formularios usan contenedor con su propio manejo (sin romper el ancho del modal).
- **Campos de texto largos auto-crecientes (`<textarea>` que crece al escribir en vertical)**: empiezan con 2–3 líneas, crecen hasta un máximo y luego hacen scroll interno. Escribir en horizontal nunca rompe el layout.
- Validación inline, mensajes de error junto al campo, foco en el primer campo, `Esc` cierra el modal (con aviso si hay cambios sin guardar), `Ctrl/Cmd+Enter` guarda.

- El modal bloquea el scroll del fondo y atrapa el foco (accesibilidad).

Documenta este patrón en `STRUCTURE.md` con una tabla "Acción → Modal o Página".

---

5. LAYOUT GLOBAL

- **Sidebar izquierdo** colapsable con los módulos (iconos + texto), **topbar** con búsqueda global (Cmd/Ctrl+K), notificaciones, accesos rápidos "+ New" y menú de usuario. Preparado para multi-organización (selector de organización, aunque solo haya una).
- Contenido en área principal con ancho máximo cómodo; en tablas, cabecera de tabla fija.
- Navegación de primer nivel: **Dashboard · CRM (con submódulos) · Projects · Finance · AI · Settings.**
- Esta fase: prioriza **escritorio** bien resuelto, pero escribe el CSS *mobile-first con breakpoints definidos en tokens* (480/768/1024/1280) para facilitar la fase posterior de responsive.

---

6. MÓDULOS (rutas y contenido)

Para cada pantalla: listado/tabla con búsqueda, filtros, orden, paginación, estados vacíos, acciones por fila y vista de detalle según el patrón híbrido. Rellena con 15–40 registros ficticios coherentes (negocio de diseño/remodelación en Miami: Coconut Grove, Brickell, Coral Gables, Miami Beach, Key Biscayne…).

6.1 Dashboard (`/`)

Panel principal con **todas las métricas del negocio**:

- KPIs: ingresos del mes/año, facturas pendientes y vencidas, estimados abiertos, proyectos activos, leads nuevos, tasa de conversión, valor del pipeline, gasto total.
- Gráficos: ingresos vs gastos mensuales, embudo del pipeline, leads por fuente (referido, Instagram, web, Google, Houzz…), proyectos por estado.
- Listas: próximas citas/visitas de la semana, tareas de hoy, últimas comunicaciones sin responder, proyectos con riesgo de presupuesto, aprobaciones pendientes.
- Salud de proyectos: avance vs presupuesto por proyecto (barras).
- No inventes deltas ni "objetivos" sin dato: si no hay dato mostrar "—".
- Selector de rango de fechas global.

6.2 CRM (`/crm/...`) — con submódulos en navegación secundaria

1. **Communications (`/crm/communications`)** — con **dos canales separados** (pestañas o sub-rutas `/whatsapp` y `/email`):
   - **WhatsApp:** UI estilo WhatsApp Web: lista de conversaciones a la izquierda (búsqueda, no leídos, asignado), hilo de mensajes en el centro (burbujas entrada/salida, hora, estado enviado/entregado/leído, adjuntos, notas internas diferenciadas, plantillas rápidas), panel derecho con ficha del contacto/lead/proyecto vinculado y acciones (crear tarea, cita, lead, ver proyecto). Indicador de "ventana de 24 h" abierta/cerrada.
   - **Email:** UI estilo Gmail: carpetas/etiquetas (Inbox, Sent, Drafts, Starred, Archive, Trash, etiquetas propias), lista de hilos con vista previa, lectura de hilo con mensajes apilados y adjuntos, **redactor** (To/Cc/Bcc, asunto, editor de texto enriquecido básico, adjuntos, plantillas, firma, guardar borrador), acciones masivas (archivar, marcar, etiquetar), vinculación a contacto/proyecto.
   - Ambos canales con la **misma estructura de datos de "conversación + mensajes"** para facilitar la conexión posterior.
   - El redactor de email y la ampliación de conversación pueden abrirse como **página** con Back; respuestas rápidas en panel inline.
2. **Agenda (`/crm/agenda`)** — calendario mes/semana/día/lista, citas y visitas de obra, colores por tipo, asignado, vinculación a contacto/proyecto, arrastrar para reprogramar, vista "Hoy". Crear/editar cita rápida = modal; detalle completo = página.
3. **Contacts (`/crm/contacts`)** — clientes, proveedores/partners, referidores, arquitectos/contratistas. Tabla + tarjetas, etiquetas, fuente, historial de comunicaciones, proyectos y facturas vinculados. Ficha completa = página.
4. **Leads (`/crm/leads`)** — alta de lead con los campos del formulario de la clienta (nombre, apellido, email, teléfono, cuándo quiere empezar, descripción del proyecto) + fuente, quién lo refirió, presupuesto estimado, tipo de servicio (diseño, remodelación, staging…), puntuación, asignado. Vista tabla y vista tarjetas.
5. **Prospecting (`/crm/prospecting`)** — listas de prospectos (arquitectos, property managers, juntas de condominio/HOA, constructoras, promotores, inmobiliarias, agentes), campañas de contacto en frío, secuencias (email/WhatsApp), estado por prospecto, métricas de respuesta, conversión a lead.
6. **Pipeline (`/crm/pipeline`)** — tablero Kanban con arrastrar y soltar. Etapas propuestas: *New Lead → Contacted → Consultation → Proposal/Estimate Sent → Negotiation → Won → Lost*. Valor por etapa, filtros, tarjeta con valor, próxima acción y días en etapa. Detalle de oportunidad = página.

6.3 Projects (`/projects`)

La clienta ve **todos sus proyectos** con métricas y gastos:

- Listado (tabla + tarjetas) con estado, cliente, dirección, fechas, presupuesto, gastado, % avance, margen.
- **Detalle de proyecto (página, con pestañas):** Overview (KPIs de presupuesto vs real, avance), Rooms/Scope (habitaciones/áreas), Tasks, Timeline/Gantt sencillo, Selections & Materials (acabados, muebles, proveedores, estado de pedido), Budget & Expenses (por categoría), Documents (planos, renders, contratos, permisos/condo approvals), Photos (antes/después, galería), Communications vinculadas, Estimates/Invoices vinculados, Notes.
- Partners/proveedores asignados al proyecto.

6.4 Finance (`/finance`)

Submódulos:

- **Estimates:** listado y **constructor de estimado** (página) con líneas (servicio/producto, cantidad, precio, impuesto, descuento), totales automáticos, términos, vista previa tipo PDF, estados (draft, sent, viewed, accepted, declined, expired), convertir en invoice.
- **Invoices:** igual estructura; estados (draft, sent, partial, paid, overdue), registro de pagos (modal), vencimientos, recordatorios.
- **Expenses / Purchase Orders:** gastos por proyecto y categoría, proveedor, recibos adjuntos, estado de aprobación.
- **Payments & Reports:** ingresos, cuentas por cobrar/pagar, rentabilidad por proyecto, resumen mensual/anual.
- Importes con formato USD, cifras tabulares, nada de redondeos silenciosos.

6.5 AI (`/ai`) — varios módulos de inteligencia artificial (solo UI y mock en esta fase)

Hub con tarjetas/subrutas:

1. **Communication Analysis:** resumen de conversaciones, sentimiento, temas recurrentes, objeciones frecuentes, tiempos de respuesta, oportunidades detectadas, conversaciones en riesgo.
2. **Prospecting Analysis:** qué segmentos/fuentes/mensajes convierten mejor, recomendaciones de siguiente acción, puntuación de prospectos.
3. **Lead Finder:** agente que **sale a buscar clientes potenciales en varias plataformas** (Google, Instagram, Houzz, Facebook, LinkedIn, directorios, sitios de bienes raíces…). UI: configurar criterios (zona, tipo de proyecto, rango de presupuesto, palabras clave), plataformas activas, ejecuciones programadas, resultados con puntuación, acción

"añadir a Leads/Prospecting". Marca claramente las fuentes y deja estado "Connect platform" simulado.

4. **Pattern Detection:** patrones de compra/conversión/estacionalidad, clientes con mayor valor de vida, mejores canales, horarios óptimos de contacto.
5. **Email Mining:** analiza los correos ya registrados en el CRM para extraer **posibles clientes** y proponer seguimiento (contactos sin lead, hilos fríos, contactos reactivables).
6. **Campaigns & Ads:** conexión (simulada) a cuentas de publicidad (Meta Ads, Google Ads), vista de campañas, gasto, leads generados, CPL/ROAS, y sugerencias de audiencia/segmentación basadas en datos del CRM. Seguimiento de leads por campaña.
7. **AI Assistant (chat):** asistente conversacional lateral/flotante con acciones sugeridas. Cada módulo: panel de configuración, historial de ejecuciones, resultados accionables y estado "datos de demostración".

6.6 Settings (`/settings`)

Organización/marca (logo, colores), usuarios y roles (admin, designer, accounting, viewer), canales conectados (WhatsApp, Email, Calendar), plantillas (email, WhatsApp, estimate, invoice), catálogo de servicios y productos, impuestos, integraciones de ads, preferencias de IA, notificaciones. Aquí vive **toda** configuración (no la repartas por la app).

---

7. CALIDAD, ACCESIBILIDAD Y RENDIMIENTO

- Navegación completa por teclado, foco visible, `aria-*` correctos en modales/tabs/menús, contraste AA.
- Sin errores ni *warnings* en consola.
- Componentes de tabla/lista con virtualización o paginación (no renderizar cientos de filas de golpe).
- Estados: cargando, vacío, error, sin permisos — en cada vista.
- Reutiliza componentes: `Button, Input, Select, Textarea(AutoGrow), DatePicker, Table, Card, Badge, Tabs, Modal/Page shells, Toast, EmptyState, Skeleton, Avatar, Kanban, Chart wrappers` .

---

8. CAPA DE DATOS (para conectar luego al backend)

- Todos los datos mock en `src/data/mock/*.js` . Ningún componente importa mock

directamente: pasan por **servicios** en `src/services/*.js` (p. ej. `leadsService.list()`, `projectsService.get(id)`, `invoicesService.create(data)`) que hoy devuelven mock (con retardo simulado) y mañana se sustituyen por llamadas a Supabase/API sin tocar la UI.

- Define en `src/data/schema.md` las entidades y campos que usa la UI (contacts, leads, prospects, opportunities/pipeline stages, projects, rooms, tasks, selections, estimates, invoices, payments, expenses, partners, communications (channel: whatsapp|email), messages, appointments, ai_runs, campaigns, users/roles). Será la base del modelo de BD.
- Hook genérico (`useResource`) para loading/error/refetch.

---

9. ORDEN DE TRABAJO SUGERIDO

1. Estructura del proyecto, tokens, layout (sidebar/topbar), router, componentes base y `FormShell/ModalShell/PageShell` con el patrón híbrido.
2. Dashboard.
3. CRM: Communications (WhatsApp y Email), Agenda, Contacts, Leads, Prospecting, Pipeline.
4. Projects.
5. Finance.
6. AI.
7. Settings.
8. Pulido: estados vacíos/carga, accesibilidad, revisión de coherencia visual, README. Haz commit por módulo.

10. ENTREGA

Cuando todo funcione en local:

1. Verifica `npm run build` y `npm run preview` sin errores.
2. Haz capturas de las pantallas principales (carpeta `docs/screenshots/`).
3. Rellena `README.md`, `DECISIONS.md` (incluye los colores/fuentes reales que sacaste de la web de la clienta y qué supuestos asumiste) y `STRUCTURE.md`.
4. Comprime la carpeta (sin `node_modules`) o súbela a un repo **privado** de GitHub si Marcos te lo indica, y envíasela a Marcos para su valoración.
   Después de su feedback, la siguiente fase es el refinamiento responsive (móvil y todas las resoluciones). La conexión a backend, base de datos y pruebas de funciones la

hacen Marcos y su asistente: **no la hagas tú**.